

Exercise 6

Checking for satisfiability

In this exercise, we're going to develop a *very* simple SAT solver. To do so, we'll first build data structures to represent boolean expressions. An expression is *SAT* if there is a consistent assignment of boolean values to the variables in the expression such that it evaluates to `True`.

The point of this exercise is not to develop a good SAT solver. We are doing the naivest possible thing.

If you want an extension that walks you through a classic algorithm, try the [2019 Haskell test](#) from Tony Field at Imperial.

The template file for this exercise is available as `sat-exercise8.hs`.

Introduction

We'll use a data type to represent our expressions

```
data Expr
  = Const Bool
  | Var String
  | Not Expr
  | And [Expr]
  | Or [Expr]
  deriving Eq
```

We implement a custom `Show` instance to pretty-print it.

The `And` and `Not` operators are n-ary, rather than binary. So they behave like `and` and `or` by accepting a list of expressions.

Our solver will be very simple, we will just find all the free variables in an expression and exhaustively try all possible assignments of boolean values to these variables, checking one such assignment results in the expression evaluating to `True`.

Evaluating an expression

Now we need a way of representing a binding of variables to boolean values, we will use a list of 2-tuples for this, and introduce a type to save on typing

```
type Bindings = [(String, Bool)]
```

Exercise 6.1

Your first job is to write a function which, given a variable name (of type `String`), looks it up in a list of bindings and returns the corresponding value. You can assume that the bindings will contain no duplicate entries (we will ensure this later). So define a function

```
find :: String -> Bindings -> Bool
```

Next we will write a function `eval` that takes an expression, and some variable bindings, and evaluates the expression:

```
eval :: Expr -> Bindings -> Bool
```

Test this function on some expressions for which you know the answer. For example:

```
Prelude> eval (Var "a") [("a", False)]
False
Prelude> eval (Or [(Var "a"), (Const True)]) [("a", False)]
True
```

Enumerating all boolean assignments

We are going to test all possible assignments until we have one that satisfies the expression, or else we run out of options (in which case the expression is not satisfiable). We need three things for this

Exercise 6.2

1. A way of extracting variables from an expression

```
vars :: Expr -> [String]
```

2. A way of generating all boolean assignments for these variables. For these we will take an integer giving the number of variables and return lists of boolean values of this length:

```
bools :: Int -> [[Bool]]
```

3. A function which glues these pieces together and returns a list of bindings.

```
bindings :: Expr -> [Bindings]
```

The `vars` function should just (similar to `eval`) destructure the expression and return the concatenation of all the variable names it finds. You may find the function `concatMap` useful. Do not worry about generating duplicate variable names in this list, we will remove them separately.

For generating all boolean lists of length n , notice that the following simple recursive approach works. If $n=0$ the list is empty. If $n>0$, the list is obtained by separately prepending both `True` and `False` to the lists obtained with $n-1$.

For constructing the list of bindings we will find all the unique variables in an expression, and zip them up with each set of boolean assignments. To build unique variables, you may find the function

```
uniquify :: Eq a => [a] -> [a]
uniquify [] = []
uniquify (x:xs) = x : filter (/= x) (uniquify xs)
```

useful.

The solver

Exercise 6.3

Now we have all the pieces in place, we can write the SAT solver itself. Our goal is to write the function:

```
sat :: Expr -> Maybe Bindings
```

If the expression is satisfiable it should return the variable assignments which result in the expression evaluating to `True`. Conversely, if the expression is not satisfiable, you should return `Nothing`.

You may find it helpful to define a helper function

```
sat' :: Expr -> [Bindings] -> Maybe Bindings
```

which iterates through the list of bindings until one of them results in the expression being satisfied.

Extension: existential and universal quantifiers

Sometimes it is convenient to write boolean expressions using [universal](#) (for all) and [existential](#) (there exists) quantifiers.

For example an expression

$$\exists a. a \wedge b$$

read as “there exists an a such that $a \wedge b$ ” is true if there is an assignment to a such that $a \wedge b$ evaluates to true.

Similarly, an expression

$\forall a. a \wedge b$

read as “for all a , $a \wedge b$ ” is true if all assignments to a result in $a \wedge b$ being true.

To handle this case we could modify our existing `data` declaration to add these new ones.

```
data Expr
  = Const Bool
  | Var String
  | Not Expr
  | And [Expr]
  | Or [Expr]
  | Exists String Expr
  | Forall String Expr
  deriving Eq
```

As discussed in the lectures, the disadvantage with this approach is that we need to go back and implement all of our functions to handle these new cases.

We will actually handle the quantifiers using *rewrite rules*. That is, we will keep our existing solver for unquantified expressions and expand out any quantification

$\exists a. e(a) \rightarrow e(\text{True}) \vee e(\text{False})$

$\forall a. e(a) \rightarrow e(\text{True}) \wedge e(\text{False})$.

We could do this eagerly, by providing functions

```
forall :: String -> Expr -> Expr
exists :: String -> Expr -> Expr
```

That construct rewritten expressions on the fly. However, this would preclude doing clever things with the quantification. Instead we will implement a new `QuantifiedExpr` type and rewriting functionality.

```
data QuantifiedExpr
  = Bare Expr
  | Exists String QuantifiedExpr
  | Forall String QuantifiedExpr
  deriving Eq
```

Again, I don't derive from `Show` because I want a pretty-printable version of the expressions.

Question 6.4

Do you understand why we need a `Bare` constructor for the “base” `Expr` we might want to hold?

Exercise 6.5

To handle problems with `QuantifiedExpr` types, we will rewrite them into plain `Expr` types. That is, we’ll write

```
rewrite :: QuantifiedExpr -> Expr
```

A function to check sat of quantified expressions is then just

```
satQuantified :: QuantifiedExpr -> Maybe Bindings
satQuantified = sat . rewrite
```

One approach is to write a helper

```
rewrite' :: QuantifiedExpr -> Bindings -> Expr
```

that takes a (possibly empty) list of bindings of variables to values and attempts to substitute them in the expression. Encountering a `Forall` or `Exists` should augment the list of bindings with ones for the quantified variable.

For example

```
rewrite' (Forall "a" (Bare (Or [Var "a", Var "b"]))) []
== (And [rewrite' (Bare (Or [Var "a", Var "b"]))) [("a", False)],
      rewrite' (Bare (Or [Var "a", Var "b"]))) [("a", True)]]
== (And [Or [Const False, Var "b"],
      Or [Const True, Var "b"]])
```

You may think of a better way of doing this.